

Lab 2, start to finish

Control flow and functions in C, written for someone who has never touched Git, GitHub, Codespaces or `make`. Follow it top to bottom and you end up with three working programs on your own GitHub page and a link to hand in.

0	Words you'll meet	2 min
1	Fork the teacher's repository	once per course
2	Open a codespace	once per course
3	Build the starter code and see what's missing	
4	Task 1: sum from 1 to n	
5	Task 2: factorial	
6	Task 3: prime numbers up to n	
7	Save your work to GitHub	
8	Commit with yesterday's date, and why that's possible	
9	Hand it in	
?	When something goes wrong	

Steps 1 and 2 you only do once; Lab 3, 4 and 5 reuse the same fork and codespace. Steps 3 to 6 are the lab itself. Everything in this PDF is clickable: the contents above, the links, and the small "contents" link at the top of each section.

About the pictures. Every dark terminal picture is a real screenshot. I ran each command on a Linux machine set up the same way as a codespace (same folder name, same Makefile, same compiler flags), typed the inputs you see, and drew the orange marks on top. GitHub's website was blocked from that machine, so the GitHub pages in steps 1 and 2 are sketches. The buttons are where the sketches show them, but the real site has more on it and looks different.

Course repository: github.com/mareksxb/Programming_Languages_C_2026_Fall_Labs

Finished files for comparison: `tutorial/lab2/friend_solution/` in the repository this PDF came from.

Words you'll meet

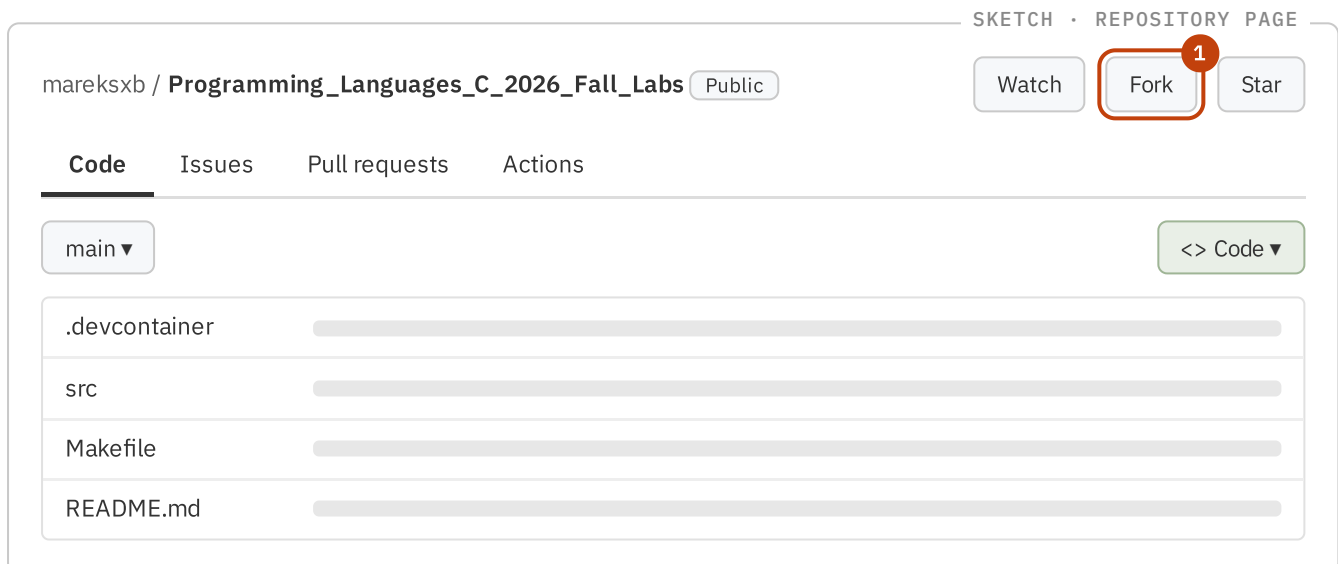
You don't need to memorise these. Come back here when a word in a later step doesn't make sense.

Repository (repo)	A project folder whose history is tracked. The teacher's repo holds the lab instructions, a Makefile and the starter <code>.c</code> files you will fill in.
Git	The program that keeps that history. It saves snapshots of the folder on request.
Commit	One saved snapshot. It records which files changed, a short message from you, your name and a date.
GitHub	A website that stores Git repositories online so other people (your teacher) can see them. Git and GitHub are different things: Git runs on a computer, GitHub is the place you upload to.
Fork	Your own copy of someone else's repository, on your GitHub account. You can change it however you like and the original doesn't notice.
Push	Upload new commits from the computer you're working on to GitHub.
Codespace	A Linux computer that GitHub runs for you in its data centre. You use it through your browser, in an editor called VS Code. The C compiler and <code>make</code> are already installed, so there's nothing to set up on your own laptop.
Terminal	The panel where you type commands instead of clicking. Each command is a program name followed by what you want it to work on, e.g. <code>ls src</code> means "list the files in the folder src".
gcc	The C compiler. It reads your <code>.c</code> text file and produces a program the computer can run. If the code has a mistake, it stops and prints an <i>error</i> ; if something just looks suspicious, it prints a <i>warning</i> and carries on.
make, Makefile	<code>make</code> reads a file called <code>Makefile</code> that lists how each program is built, then types the long <code>gcc</code> command for you. It also skips work that's already done: if a <code>.c</code> file hasn't changed since the last build, <code>make</code> leaves it alone.

Fork the teacher's repository

You can read the teacher's repository but you can't change it. A fork gives you a copy under your own name, so your commits have somewhere to live and the teacher can look at all of your labs through one link.

1. Sign in at github.com. If you don't have an account, create one (free). A student email gets you the free Student Developer Pack, which includes more codespace hours, but any email works.
2. Open github.com/mareksxb/Programming_Languages_C_2026_Fall_Labs.
3. Click **Fork** near the top right (mark 1 below).
4. On the "Create a new fork" page, leave the owner as yourself and the name as it is. Keep "Copy the main branch only" ticked. Click **Create fork** (mark 2).
5. A few seconds later you're on github.com/your-name/Programming_Languages_C_2026_Fall_Labs. Under the name it says "forked from mareksxb/...". That line is how you can always tell you're in your copy and not the teacher's.

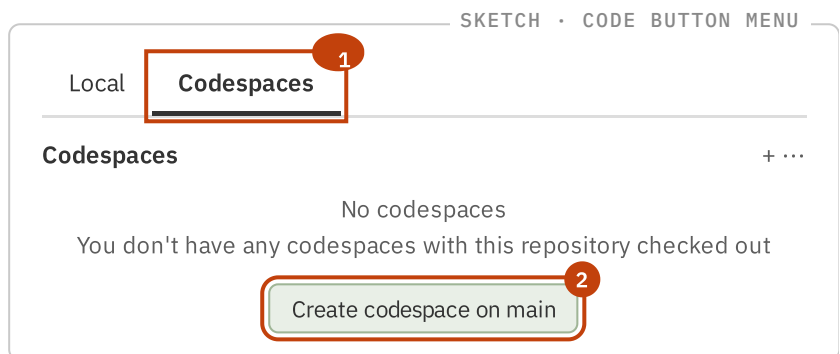


Later in the course the teacher adds new files (Week 4, Week 5). Your fork then shows "This branch is N commits behind". Click **Sync fork**, then **Update branch**, and in your codespace run `git pull` to get them.

Open a codespace

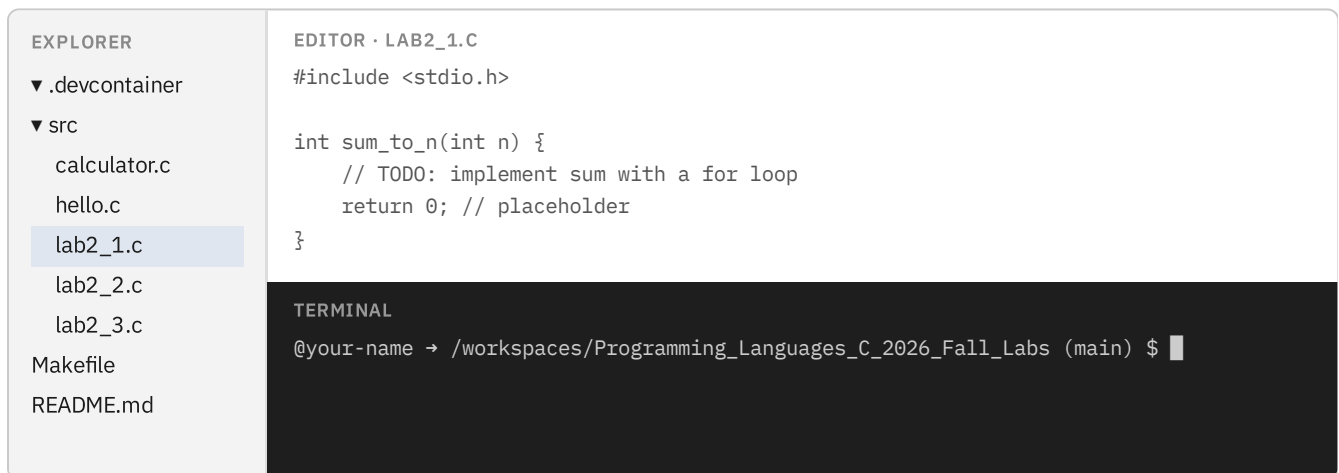
To turn C code into a program you need a compiler, and installing one on Windows is a chore. A codespace sidesteps that: it's a Linux machine in GitHub's data centre with the compiler already on it, and your browser is the screen and keyboard. It works the same from Windows, a Mac or a library PC.

1. Go to **your fork**. Check the top left says *your-name/Programming_Languages...*, not *mareksxb*.
2. Click the green **<> Code** button and switch to the **Codespaces** tab (mark 1).
3. Click **Create codespace on main** (mark 2). The first start takes a couple of minutes. The repository has a `.devcontainer` folder that tells GitHub what to install, and when setup finishes it runs `make` once on its own, so you'll see compiler output scroll past. That's expected.



What you'll see

The browser tab turns into VS Code. Three areas matter for this lab:



Sketch of the codespace layout. Your terminal prompt looks like the one in the sketch; my screenshots use a shorter prompt.

Explorer (left). Your files. Click `src`, then a file, to open it.

Editor (top). Type your code here and press `Ctrl+S` (`Cmd+S` on Mac) to save.

Terminal (bottom). Commands go here. Missing? Press `Ctrl+`` or use the menu: Terminal, New Terminal.

A codespace stops by itself after about 30 idle minutes and keeps your files. To return, open your fork, click **Code**, **Codespaces**, and pick the one listed there instead of creating a new one. Free accounts get a limited number of codespace hours a month, so one codespace for the whole course is plenty.

One surprise: the repo's settings reformat a file whenever you save it ("format on save", Google style), so indentation may jump from 4 spaces to 2. Nothing is broken; it's only spacing.

Build the starter code and see what's missing

Before writing anything, build the teacher's untouched `lab2_1.c` and run it. Click into the terminal and type the two commands below, pressing `Enter` after each. When the program asks for a number, type `10` and press `Enter`.

```
$ ls src
$ make bin/lab2_1 && ./bin/lab2_1
```

1 The function never uses `n`. It's still the teacher's empty placeholder.

2 Nobody checks whether `scanf` actually read a number.

3 You type 10, press Enter... and nothing is printed. `main()` stops right after `scanf`.

The teacher's starter file, built and run without changes.

It compiles, but typing 10 does nothing. The two warnings say why: the function ignores its input, and nobody checks what `scanf` read. Filling those gaps is the whole lab.

What that command line means

- `make bin/lab2_1` asks make to produce the file `bin/lab2_1`. The Makefile has a rule saying "`bin/NAME` is built from `src/NAME.c`", so make runs the `gcc` line you see in the screenshot. `-Wall -Wextra -Wpedantic` switch on extra warnings, and `-o bin/lab2_1` names the result.
- `&&` means "only if the first command succeeded". If your code has an error, the build fails and the old program doesn't start by mistake.
- `./bin/lab2_1` runs the program. `./` means "starting from the folder I'm in".

The command in the lab text doesn't work. The lab text says `make lab2_1`. With this Makefile that prints No rule to make target 'lab2_1', because the Makefile names its programs `bin/lab2_1`. Use `make bin/lab2_1` for one program or `make lab2` for all three. Running them is the same: `./bin/lab2_1`.

The loop for every task from now on: open the file in the Explorer, edit, save with `Ctrl + S`, then in the terminal press `↑` to bring back the last command and `Enter` to run it again. If `make` says '`bin/lab2_1`' is up to date, you forgot to save.

Task 1: sum from 1 to n

WHAT THE TEACHER ASKS

- Write `int sum_to_n(int n)` that adds $1 + 2 + \dots + n$ with a `for` loop.
- In `main`: ask for `n`. If `n < 1`, print an error. Otherwise call `sum_to_n` and print the result.

Here is the whole file after the change. Grey crossed-out lines are the teacher's placeholders that I removed; green lines are new. Everything else is untouched.

```
#include <stdio.h>
/* ... task description comment, unchanged ... */

int sum_to_n(int n) {
- // TODO: implement sum with a for loop
- return 0; // placeholder
+ int total = 0; /* running sum, starts empty */
+
+ for (int i = 1; i <= n; i++) { /* i takes the values 1, 2, ..., n */
+     total = total + i;
+ }
+
+ return total;
}

int main(void) {
    int n;

    printf("Enter a positive integer n: ");
- scanf("%d", &n);
+ if (scanf("%d", &n) != 1) {
+     printf("Input error: please type a whole number.\n");
+     return 1;
+ }

- // TODO: validate input, call function, and print result
+ if (n < 1) {
+     printf("Error: n must be 1 or bigger (you typed %d).\n", n);
+     return 1;
+ }
+
+ int result = sum_to_n(n);
+ printf("1 + 2 + ... + %d = %d\n", n, result);

    return 0;
}
```

Legend: added removed

The function

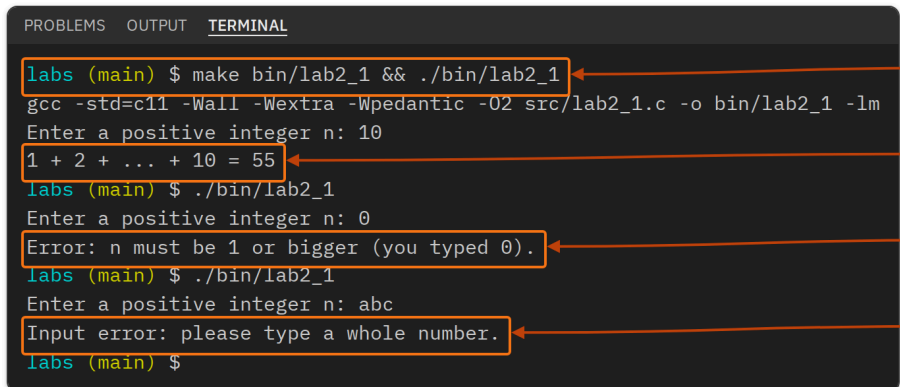
`int sum_to_n(int n)` reads as: a function called `sum_to_n` that takes one whole number, calls it `n`, and gives back a whole number. The value after `return` is what it gives back.

A `for` loop has three parts in brackets, separated by semicolons: where to start (`int i = 1`), when to keep going (`i <= n`), and what to do after each round (`i++`, add one to `i`). For `n = 4` the body runs four times, with `i = 1, 2, 3, 4`, and `total` grows 1, 3, 6, 10.

Scope, one of the lab's goals: `total` and `i` exist only inside `sum_to_n`, and `main` can't see them. The `n` inside the function is also a separate variable from the `n` in `main`. When `main` calls `sum_to_n(n)`, C copies the value across, so changing `n` inside the function would leave `main`'s copy alone.

main

`scanf` returns how many values it managed to read. Type letters and it returns 0, leaving `n` with a random leftover value. Checking `!= 1` catches that; it's the same check as in `calculator.c` from Lab 1. Next, `if (n < 1)` rejects zero and negative numbers. `return 1` from `main` tells the system "finished with an error", `return 0` means "finished fine".



```
PROBLEMS OUTPUT TERMINAL
labs (main) $ make bin/lab2_1 && ./bin/lab2_1
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 src/lab2_1.c -o bin/lab2_1 -lm
Enter a positive integer n: 10
1 + 2 + ... + 10 = 55
labs (main) $ ./bin/lab2_1
Enter a positive integer n: 0
Error: n must be 1 or bigger (you typed 0).
labs (main) $ ./bin/lab2_1
Enter a positive integer n: abc
Input error: please type a whole number.
labs (main) $
```

1 make compiles `src/lab2_1.c` into `bin/lab2_1`, then `&&` starts it

2 `1 + 2 + ... + 10 = 55`. Correct.

3 `n < 1`, so we print an error instead

4 Letters instead of a number: `scanf` fails and we catch it

Three runs: a normal one, one with `n = 0`, one with letters.

Task 2: factorial

WHAT THE TEACHER ASKS

- Write `long long factorial(int n)` that computes $n!$ with a loop. No recursion (the function must not call itself).
- In `main`: ask for n . If $n < 0$, print an error. Otherwise call `factorial` and print the result.

```
#include <stdio.h>
/* ... task description comment, unchanged ... */

long long factorial(int n) {
- // TODO: compute factorial iteratively
- return 1; // placeholder
+ long long result = 1;          /* 0! and 1! are both 1 */
+
+ for (int i = 2; i <= n; i++) { /* multiply 2 * 3 * ... * n */
+     result = result * i;
+ }
+
+ return result;
}

int main(void) {
    int n;

    printf("Enter a non-negative integer n: ");
- scanf("%d", &n);
+ if (scanf("%d", &n) != 1) {
+     printf("Input error: please type a whole number.\n");
+     return 1;
+ }

- // TODO: validate input, call function, print result
+ if (n < 0) {
+     printf("Error: factorial is not defined for negative numbers.\n");
+     return 1;
+ }
+
+ if (n > 20) {
+     /* 21! does not fit into a long long (max is about 9.2 * 10^18) */
+     printf("Error: n must be 20 or less, otherwise the result overflows.\n");
+     return 1;
+ }
+
+ printf("%d! = %lld\n", n, factorial(n));

    return 0;
}
```

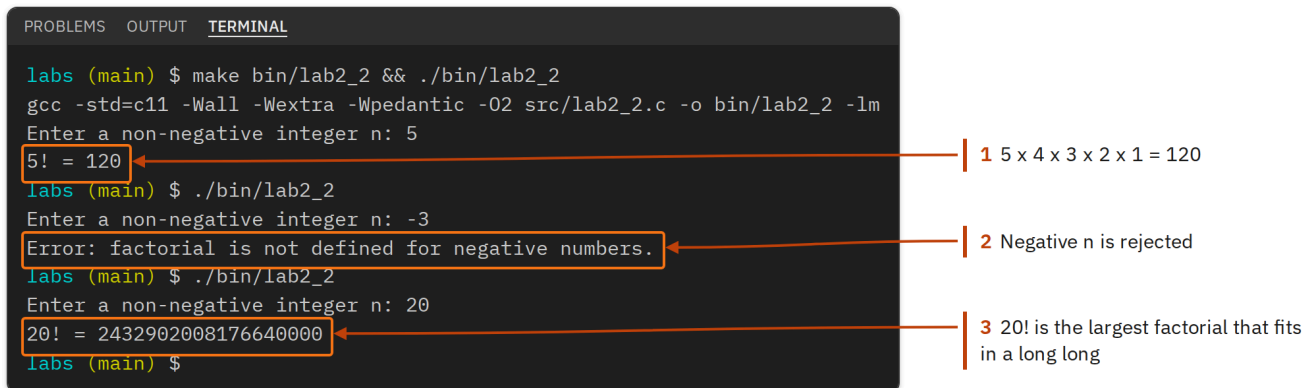
How it works

$n!$ means $1 \times 2 \times \dots \times n$. `result` starts at 1, not 0, because anything multiplied by 0 stays 0. The loop starts at 2 since multiplying by 1 changes nothing. For $n = 0$ or $n = 1$ the loop never runs and the function returns 1, which is the correct answer for both ($0! = 1$ by definition).

Why long long

Factorials grow very fast. An `int` holds numbers up to 2,147,483,647, and $13! = 6,227,020,800$ is already past that. `long long` goes to about 9.2×10^{18} , which fits $20!$ but not $21!$ (51,090,942,171,709,440,000). To print a `long long` you need `%lld` in `printf`; plain `%d` would print garbage.

The `n > 20` check isn't in the task. I added it because without it, 21 and above print a wrong number with no warning. If you want to match the task exactly, delete that `if` block (the five lines with the overflow message). The rest works the same.



The image shows a terminal window with the following content:

```
labs (main) $ make bin/lab2_2 && ./bin/lab2_2
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 src/lab2_2.c -o bin/lab2_2 -lm
Enter a non-negative integer n: 5
5! = 120
labs (main) $ ./bin/lab2_2
Enter a non-negative integer n: -3
Error: factorial is not defined for negative numbers.
labs (main) $ ./bin/lab2_2
Enter a non-negative integer n: 20
20! = 2432902008176640000
labs (main) $
```

Annotations on the right side of the terminal output:

- 1 $5 \times 4 \times 3 \times 2 \times 1 = 120$ (points to `5! = 120`)
- 2 Negative n is rejected (points to `Error: factorial is not defined for negative numbers.`)
- 3 20! is the largest factorial that fits in a long long (points to `20! = 2432902008176640000`)

5!, a negative input, and the largest value that fits.

Task 3: prime numbers up to n

WHAT THE TEACHER ASKS

- Write `int is_prime(int n)` that returns 1 if n is prime and 0 if not.
- In `main`: ask for n. If $n < 2$, print an error. Otherwise print every prime $\leq n$.

```
#include <stdio.h>
/* ... task description comment, unchanged ... */

int is_prime(int n) {
- // TODO: check if n is prime using loop up to sqrt(n)
- return 0; // placeholder
+ if (n < 2) {
+     return 0;          /* 0, 1 and negatives are not prime */
+ }
+
+ /* Try every divisor d while d * d <= n, i.e. d <= sqrt(n).
+    If n had a divisor bigger than sqrt(n), it would also have
+    a matching one smaller than sqrt(n), so we would find that first. */
+ for (int d = 2; d * d <= n; d++) {
+     if (n % d == 0) {
+         return 0;      /* found a divisor, so not prime */
+     }
+ }
+ return 1;
}

int main(void) {
    int n;

    printf("Enter an integer n (>= 2): ");
- scanf("%d", &n);
+ if (scanf("%d", &n) != 1) {
+     printf("Input error: please type a whole number.\n");
+     return 1;
+ }

- // TODO: validate input and print all primes up to n
+ if (n < 2) {
+     printf("Error: n must be at least 2.\n");
+     return 1;
+ }
+
+ printf("Primes up to %d:\n", n);
+ for (int k = 2; k <= n; k++) {
+     if (is_prime(k)) {
+         printf("%d ", k);
+     }
+ }
+ printf("\n");

    return 0;
}
```

is_prime

A prime has exactly two divisors: 1 and itself. The function tries $d = 2, 3, 4, \dots$ and checks $n \% d$, the remainder of n divided by d . A remainder of 0 means d divides n evenly, so n isn't prime and the function returns 0 on the spot. If the loop ends without finding a divisor, it returns 1.

The teacher's comment says to loop "up to $\sqrt{n}$ ". Divisors come in pairs: $36 = 2 \times 18 = 4 \times 9 = 6 \times 6$. In every pair one number is at most $\sqrt{36} = 6$. So if nothing up to $\sqrt{n}$ divides n , nothing above it will either. $d * d \leq n$ means the same as $d \leq \sqrt{n}$ but stays with whole numbers, so there's no need for `math.h` or rounding worries.

main

After checking the input, a second loop walks $k = 2, 3, \dots, n$ and prints k whenever `is_prime(k)` returns 1. Here a function you wrote gets called from somewhere else, and `main` doesn't need to know how it works inside.

```
PROBLEMS OUTPUT TERMINAL
labs (main) $ make bin/lab2_3 && ./bin/lab2_3
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 src/lab2_3.c -o bin/lab2_3 -lm
Enter an integer n (>= 2): 50
Primes up to 50:
2 3 5 7 11 13 17 19 23 29 31 37 41 43 47
labs (main) $ ./bin/lab2_3
Enter an integer n (>= 2): 1
Error: n must be at least 2.
labs (main) $
```

1 Every k from 2 to 50 for which `is_prime(k)` returned 1

2 $n < 2$ is rejected

Primes up to 50, then an input that's too small.

Build all three at once

`make lab2` builds only the Lab 2 programs. Plain `make` builds every lab whose file exists, including the Lab 3 starters, which print warnings until you do Lab 3. Warnings don't stop the build.

```
PROBLEMS OUTPUT TERMINAL
labs (main) $ make clean
rm -rf bin
labs (main) $ make lab2
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 src/lab2_1.c -o bin/lab2_1 -lm
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 src/lab2_2.c -o bin/lab2_2 -lm
gcc -std=c11 -Wall -Wextra -Wpedantic -O2 src/lab2_3.c -o bin/lab2_3 -lm
labs (main) $ ls bin
lab2_1 lab2_2 lab2_3
labs (main) $
```

1 Deletes the whole `bin/` folder

2 Builds all three Lab 2 programs in one go

3 The finished programs live in `bin/`

Save your work to GitHub

Saving a file with `Ctrl+S` only saves it inside the codespace. The teacher sees nothing until you commit and push. Run these in the terminal:

```
# which files changed?
$ git status
# choose what goes into the snapshot
$ git add src/lab2_1.c src/lab2_2.c src/lab2_3.c
# take the snapshot, with a message
$ git commit -m "Lab 2 done"
# upload it to your fork on GitHub
$ git push
```

```
PROBLEMS OUTPUT TERMINAL
labs (main) $ date
Thu Oct 1 09:37:10 UTC 2026
labs (main) $ git status --short
M src/lab2_1.c
M src/lab2_2.c
M src/lab2_3.c
labs (main) $ git add src/lab2_1.c src/lab2_2.c src/lab2_3.c
labs (main) $ git commit -m "Lab 2 done"
[main 31c6f68] Lab 2 done
3 files changed, 71 insertions(+), 12 deletions(-)
labs (main) $ git log -1 --format=fuller
commit 31c6f68dd45474b9d6f66f6b58cd5c50bb90b42c (HEAD -> main)
Author: Your Friend <friend@example.com>
AuthorDate: Thu Oct 1 09:37:12 2026 +0000
Commit: Your Friend <friend@example.com>
CommitDate: Thu Oct 1 09:37:12 2026 +0000

    Lab 2 done
labs (main) $
```

1 The computer's clock. Codespaces machines run on UTC.

2 M = modified, but not committed yet

3 Commit created

4 AuthorDate and CommitDate: git copied both from the clock

The first three commands and a look at the result. `git push` isn't shown because this machine had no access to GitHub.

- `git add` names the files to include. I list the three files by name so nothing else sneaks in. The compiled programs in `bin/` are excluded anyway by the repo's `.gitignore`.
- `-m "Lab 2 done"` is the message people see next to the commit on GitHub.
- In a codespace you're already signed in, so `git push` doesn't ask for a password. If it says `Permission denied`, you opened the codespace on the teacher's repo instead of your fork; see [troubleshooting](#).

Prefer clicking? The **Source Control** icon in the left bar (three dots joined by lines) does the same: type the message, click **Commit**, then **Sync Changes**. VS Code may ask whether to stage all changes; say yes.

Afterwards, refresh your fork in the browser. "Lab 2 done" now appears next to the `src` folder.

Commit with yesterday's date

Use this *instead of* the plain `git commit` from step 7. It sets the commit's date to **30 September 2026, 23:40** in our time zone (UTC+3):

```
$ GIT_AUTHOR_DATE="2026-09-30T23:40:00+03:00" \  
  GIT_COMMITTER_DATE="2026-09-30T23:40:00+03:00" \  
  git commit -m "Lab 2 done"  
# check: AuthorDate and CommitDate should both read Wed Sep 30 23:40:00 2026  
+0300  
$ git log -1 --format=fuller  
$ git push
```

Already ran the normal `git commit` but haven't pushed yet? Change the date of that last commit instead of making a new one:

```
$ GIT_COMMITTER_DATE="2026-09-30T23:40:00+03:00" \  
  git commit --amend --no-edit --date="2026-09-30T23:40:00+03:00"
```

Copy the command exactly, quotes included. The `\` at the end of a line means "the command continues on the next line"; you can also type it all on one line without the backslashes.

Why this works

A commit is a small text record, and you can print one as it's stored:



```
PROBLEMS  OUTPUT  TERMINAL  
  
labs (main) $ git cat-file -p HEAD  
tree 5359da6a5bd71657e09964787ebc751ddc21195e  
parent 9f7a6eae168a989a03bb86124868f72ba8036208  
author Your Friend <friend@example.com> 1790847432 +0000  
committer Your Friend <friend@example.com> 1790847432 +0000  
  
Lab 2 done  
labs (main) $
```

- 1 Name, email, then the time as a plain number: seconds since 1 January 1970
- 2 Same thing for the committer. This is everything git knows about time.

`git cat-file -p HEAD` prints the latest commit exactly as git stored it.

The time is just a number: seconds since 1 January 1970 (UTC), then the time zone. When you run `git commit`, git reads that number from the computer's clock and writes it in. Nobody checks it. Git works entirely on your machine, even with no internet, and GitHub only receives the finished record later when you push. If the variables `GIT_AUTHOR_DATE` and `GIT_COMMITTER_DATE` are set, git takes the time from them instead of the clock. With the command above, both lines would end in `1790800800 +0300`, which is 23:40 on 30 September in UTC+3.

- **Two dates.** Each commit has an *author* date (when the change was written) and a *committer* date (when it was added to the history). They differ when you amend or someone else applies your work.

The `--date` option only changes the author date, so the command sets both through variables to keep them identical.

- The `NAME=value command pattern` sets a variable for that one command only. The next command uses the real clock again, so later commits aren't affected.
- The `+03:00` matters. Codespaces run on UTC (the `date` output in step 7 shows it). Leave the time zone off and git reads 23:40 as UTC, which is 02:40 on 1 October in our time.

What it doesn't hide. This changes the date stored in the commit, and that's the date GitHub shows in the commit list. GitHub still records when the push actually arrived. Anyone with access to the repository can see that under its **Activity** view and in GitHub's API, and the codespace's creation time is logged too. So the commit shows the date you chose, but the push time is still on record.

Hand it in

1. Open your fork on GitHub, click into `src` and open `lab2_1.c`. If you see your code and not the TODOs, the push worked. Do the same for the other two.
2. Copy the address from the browser bar: `https://github.com/your-name/Programming_Languages_C_2026_Fall_Labs`. Send that link. It's the option the teacher prefers.
3. A fork of a public repository is public, so the teacher can open it without doing anything. Only if you made your own private copy: **Settings, Collaborators, Add people**, then invite mareksxb or PeterisVL. Personal accounts don't offer a read-only role, so the normal invite is fine.
4. The other allowed option is uploading the three files. In the codespace Explorer, right-click each file and choose **Download**.

Before you send it

- ☐ `make lab2` finishes with no errors and no warnings for lab2 files.
- ☐ `./bin/lab2_1` with 10 prints 55; with 0 prints an error.
- ☐ `./bin/lab2_2` with 5 prints 120; with -3 prints an error.
- ☐ `./bin/lab2_3` with 30 prints 2 3 5 7 11 13 17 19 23 29; with 1 prints an error.
- ☐ `git status` says "nothing to commit, working tree clean" and "up to date with 'origin/main'".
- ☐ The three files on github.com show your code.

When something goes wrong

YOU SEE

WHAT TO DO

No rule to make target 'lab2_1'

Use `make bin/lab2_1` (see [step 3](#)).

'bin/lab2_1' is up to date, but your change isn't there

The file isn't saved. A dot or circle on the editor tab means unsaved. Press `Ctrl + S`.

error: expected ';' before ...

A missing semicolon, usually at the end of the line *above* the one gcc names.

The program just sits there

It's waiting for you to type a number and press `Enter`.
`Ctrl + C` stops a program.

Permission denied or 403 on `git push`

The codespace belongs to the teacher's repo. Commit there, run `git remote set-url origin https://github.com/your-name/Programming_Languages_C_2026_Fall_Labs.git`, then push again. Next time, create the codespace from your fork.

rejected ... (fetch first) on push

GitHub has commits you don't, e.g. after Sync fork. Run `git pull`, then `git push`.

Please tell me who you are

`git config --global user.name "Your Name"` and `git config --global user.email "you@example.com"`, then commit again.